Mobile Application Development (MAP711S)

Generics and Collections

Simon H. Muchinenyika

Generics and Collections

Table of Contents

- Generics Introduction
  - Introduction, Constraints, Type Projections
- Examples
  - Limitations, Opportunities
- Collections Introduction
  - Anatomy, Iterable, MutableIterable
- Collection Anatomy
  - Operations, Examples, Arrays, Ranges, Sequence, List

Generics and Collections

Generics Introduction

```
fun quickSort(collection: CollectionOfInts) { ... }
quickSort(listOf(1, 2, 3)) // OK
quickSort(listOf(1.0, 2.0, 3.0)) // NOT OK
```

```
fun quickSort(collection: CollectionOfDoubles) { ... } // overload
quickSort(listOf(1.0, 2.0, 3.0)) // OK
quickSort(listOf(1, 2, 3)) // OK
```

Kotlin Number inheritors: Int, Double, Byte, Float, Long, Short

Do we need 4 more implementations of quickSort?

Generics and Collections

Generic Algorithms

- Quicksort doesn't care what it is sorting, as long as it can compare two values.

```
fun <T : Comparable<T>> quickSort(collection: Collection<T>): Collection<T> { ... }

quickSort(listOf(1.0, 2.0, 3.0)) // OK

quickSort(listOf(1, 2, 3)) // OK

quickSort(listOf("one", "two", "three")) // OK
```

- generics are written in angle brackets <T>, where T is a placeholder of a type.

Generics and Collections

Another Example

- consider a holder type: a box which can hold a value of any type.

```
class Holder<T>(val value: T) { ... }

val intHolder = Holder<Int>(23)

val cupHolder = Holder("cup") // Generic parameter type can be inferred
```

- a type can be explicitly written in the angle brackets, or
- it can be inferred.

Generics and Collections

Constraints

- if we don't want to work with an arbitrary type, & we expect some functionality, we use type constraints in the form of upper bounds.

```
class Pilot<T : Movable>(val vehicle: T) {
  fun go() { vehicle.move() }
}

val tuliHafeni = Pilot<Car>(Car("Toyota", "Prado"))
val captainNyandoro = Pilot<Plane>(Plane("Airbus", "A320"))
```

- for the quickSort, the type T has to be comparable for it to work.
- upper bounds describe the supertypes that type parameter T must definitely have.
- in the example, objects are expected to pilot something movable (T: Movable).

Generics and Collections

Several Type Parameters

- just like regular functions, generic code can have several parameters:

```
public interface MutableMap<K, V> : Map<K, V> { ... }
```

- this means we can have several constraints that should all be implemented by a parameter:

```
fun <T, S> moveInAnAwesomeWayAndCompare(a: T, b: S) where T : Comparable<T>, S :
Comparable<T>, T : Awesome, T : Movable { ... }
```

- use a star-projection, * (Any? / Nothing) if you do not care about the parameter type:

```
fun printKeys(map: MutableMap<*, *>) { ... }
```

Generics and Collections

Kotlin Type Projections

Revisit:

```
interface Holder<T> {
  fun push(newValue: T) // consumes an element

    fun pop(): T // produces an element

    fun size(): Int // does not interact with T
```

Type projections in Kotlin:

```
G<T> // invariant, can consume and produce elements
G<in T> // contravariant, can only consume elements
G<out T> // covariant, can only produce elements
G<*> // star-projection, does not interact with T
```

Generics and Collections

Subtyping in Generics

```
open class A
open class B : A()      —--->  Nothing <: C <: B <: A <: Any
class C : B()

class Holder<T>(val value: T) { ... }

Holder<Nothing> ??? Holder<C> ??? Holder<B> ??? Holder<A> ??? Holder<Any>
```

Is it possible to establish any subtyping between different versions of this type?

Generics and Collections

Generic Are Invariant

Holder<Nothing> ~~<:~~ Holder<C> ~~<:~~ Holder<B> ~~<:~~ Holder<A> ~~<:~~ Holder<Any>

- there is no subtyping relationship between different versions of an invariant generic type.

```
val c: C = C()
val b: B = c // C <: B, OK
```

Vs.

```
val holderC: Holder<C> = Holder(C())
val holderB: Holder<B> = holderC // ERROR: Type mismatch. Required: Holder<B>. Found: Holder<C>.
```

- but

```
val holderB: Holder<B> = holder(C()) // OK because of casting
```

Generics and Collections

Type Project: in

```
class Holder<T> (var value: T?) {
   ...
   fun gift(other: Holder<in T>) { other.push(pop()) }
}
holderB.gift(holderA) // OK
```

- only methods that **accept** the type parameter T are called, i.e., those you can push().
- This is contravariance:

Nothing <: C <: B <: A <: Any
Holder<Nothing> :> Holder<C> :> Holder<B> :> Holder<A> :> Holder<Any>

Generics and Collections

Type Project: out

```
class Holder<T> (var value: T?) {
  ...
  fun steal(other: Holder<out T>) { value = other.pop() }
}
holderA.steal(holderB) // OK
```

- only methods that **return** a type parameter T are called, i.e., those you can pop().
- This is covariance:

  Nothing <: C <: B <: A <: Any
  Holder<Nothing> <: Holder<C> <: Holder<B> <: Holder<A> <: Holder<Any>

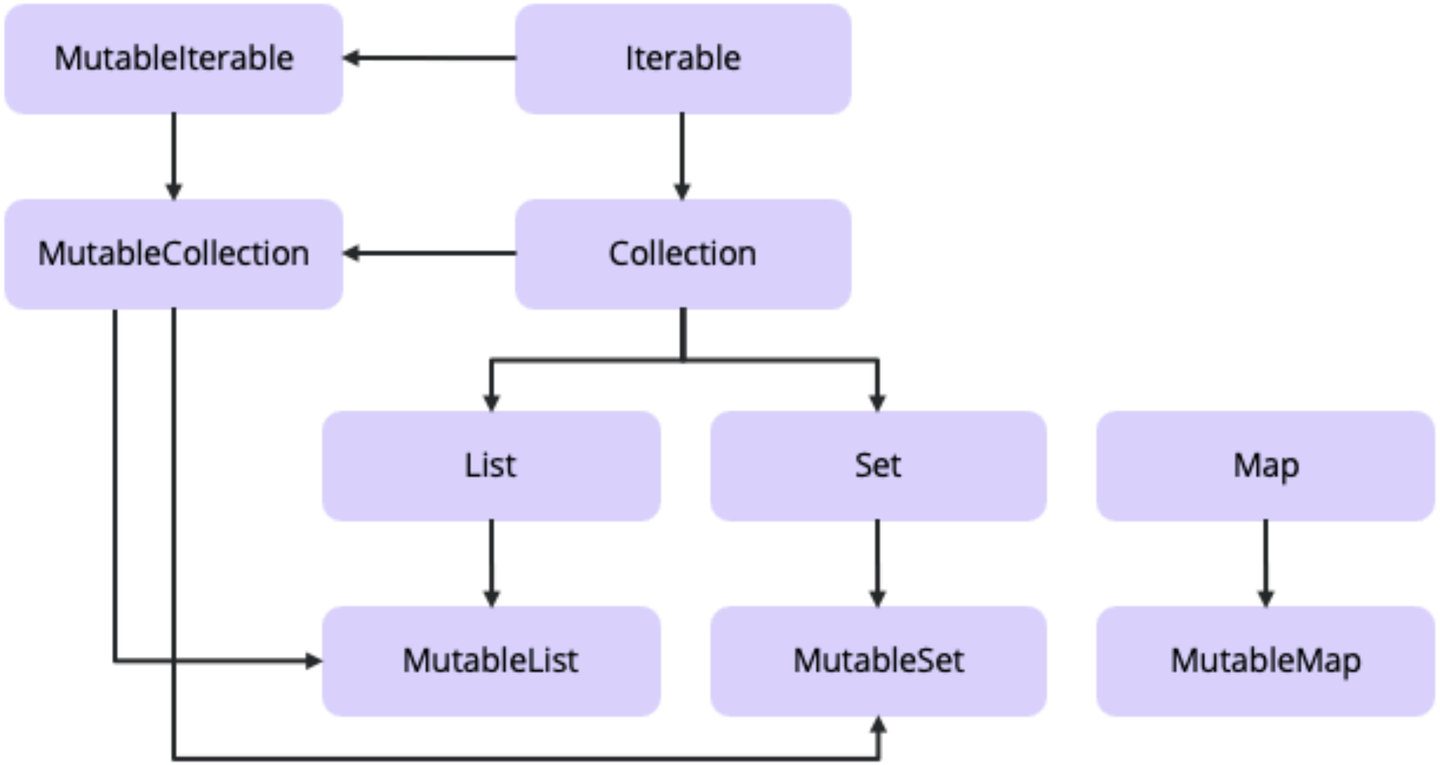
Generics and Collections

Collections Introduction

- a collection contains a number of objects of the same type.
- objects in a collection are called elements or items:
  - Lists are ordered collections with elements accessible by indices.
  - Sets are collections of unique elements.
  - Maps or dictionaries are sets of key-value pairs. The keys are unique, mapping to exactly one value, while the values can be duplicated.
- the Kotlin standard library offers generic interfaces, classes, and functions for creating, populating, and managing collections of any type.

Generics and Collections

Collections Taxonomy



Generics and Collections

Iterable

- all collections in Kotlin implement Iterable interface:

```
public interface Iterable<out T> {
    // Returns an iterator over the elements of this object.
   public operator fun iterator(): Iterator<T>
}
```

- all collections are Iterable.

```
val iterator = myIterableCollection.iterator()
while (iterator.hasNext()) {
   iterator.next()
}
```

Generics and Collections

Types of Collections

- Collection & MutableCollection are the two kinds of collections.
- Collection implements only the Iterable interface, while MutableCollection implements the Collection & MutableIterable interfaces.

```
val readonlyCollection = listOf(1, 2, 3)
readonlyCollection.add(4) // ERROR: Unresolved reference: add

val mutableCollection = mutableListOf(1, 2, 3)
mutableCollection.add(4) // OK
```

Generics and Collections

Collection Anatomy

Projection. Accepts all inheriting types as elements.

Abstraction

Parent generic

```
interface List<out E>: Collection<E>
```

Type name

Generic type name

Parent type name

Generics and Collections

Collection Example: List

```
val list1 = emptyList<Int>() // Builds the internal object EmptyList
val list2 = listOf<Int>() // Calls emptyList()
val list3 = listOf(1, 2, 3) // The type can be inferred

val list4 = mutableListOf<Int>() // But better: ArrayList<Int>()
val list5 = mutableListOf(1, 2, 3) // The type can be inferred
val list6 = buildList {
    // constructs MutableList<Int>
    add(5)
    addAll(0, listOf(1, 2, 3))
}
```

Generics and Collections

Collection Example: Set

```
public interface Set<out E> : Collection<E> {
  abstract val size: Int

  abstract fun contains(element: @UnsafeVariance E): Boolean

    abstract fun containsAll(collection: Collection<E>): Boolean

    abstract fun isEmpty(): Boolean

    abstract fun iterator(): Iterator<E>
}
```

Generics and Collections

Further Study

- Subtyping: https://en.wikipedia.org/wiki/Subtyping

Generics and Collections

References

- Generics: https://kotlinlang.org/docs/generics.html
- Iterators: https://kotlinlang.org/docs/iterators.html

Generics and Collections

Thank You

Questions?

T: +264 61 207 2054

F: +264 61 207 9054

E: smuchinenyika@nust.na